MongoDB and Node.js

Accessing MongoDB using Node.js

MongoDB Structure

- **Project**: contains one or many clusters
- **Cluster**: a set of databases
- **Database**: a set of collections (like directories)
- **Collection**: like a table (a directory with files)
- **Document**: like a row in a table (a file)

MongoDB Cluster Structure

A MongoDB cluster is a group of MongoDB servers working together as a single system.

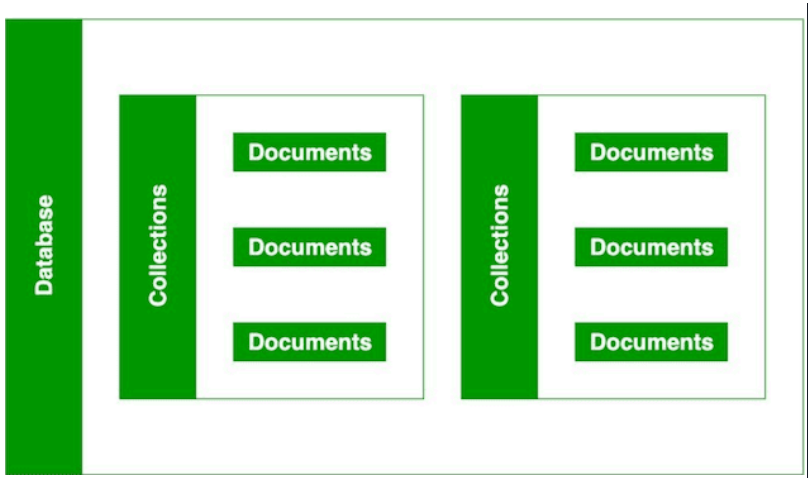
- You access it with a connection string like:

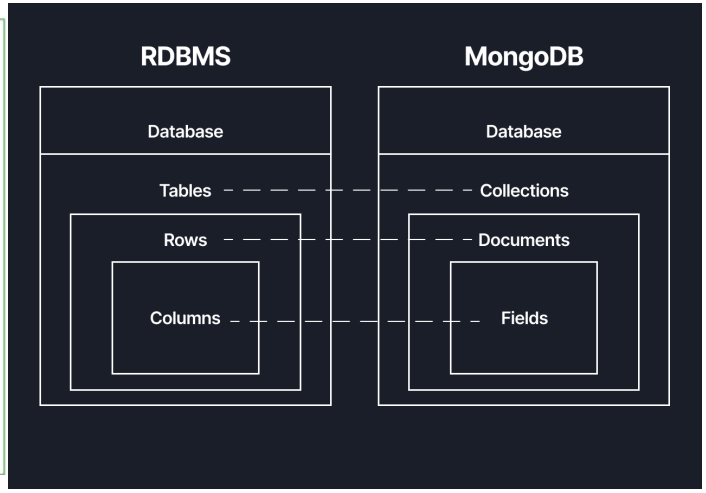
```
mongodb+srv://user:pass@cluster0.xxxxx.mongodb.net
```

A MongoDB Cluster provides reliability, safety, scalability, and performance that a single machine cannot deliver.

MongoDB Database

One Database has multiple collections that have multiple documents.





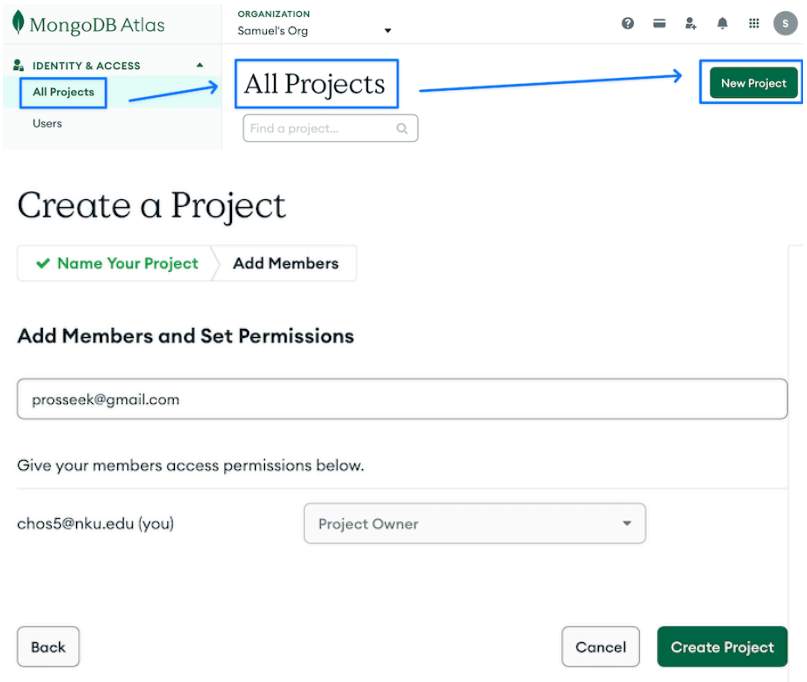
- A table in an RDBMS is equivalent to a collection (directory).
- A row in a table is equivalent to a document (one file).

Using MongoDB Atlas

The simplest way to use MongoDB is to use Atlas and Compass.

1. Register for MongoDB Atlas:
   https://www.mongodb.com/products/platform
2. Use Compass, a standalone desktop app, to access MongoDB:
   https://www.mongodb.com/products/tools/compass

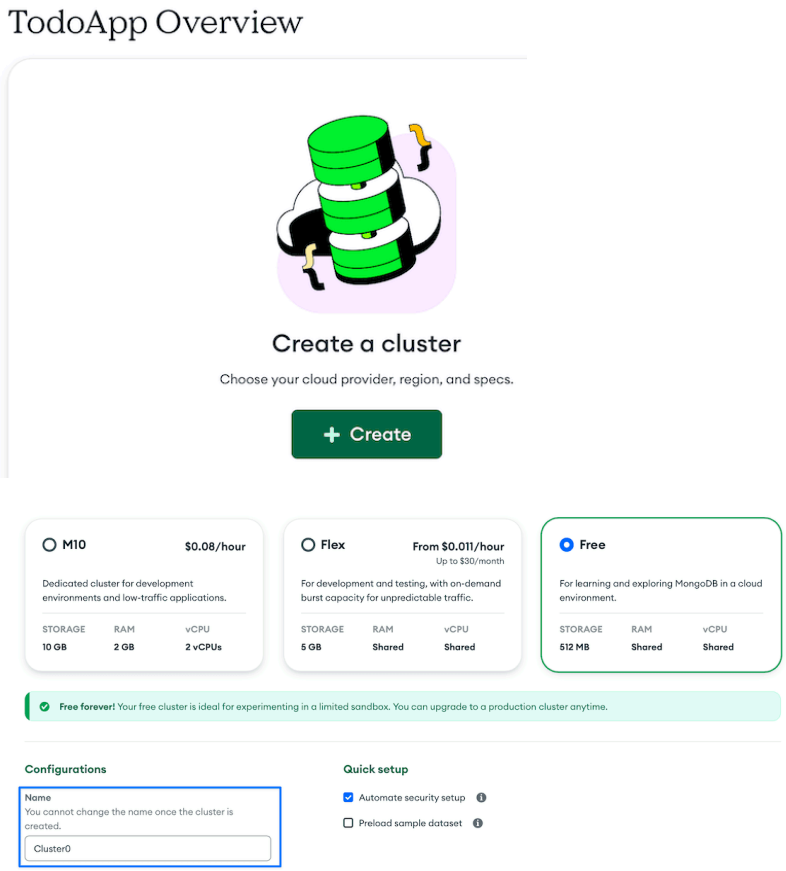
Create Project



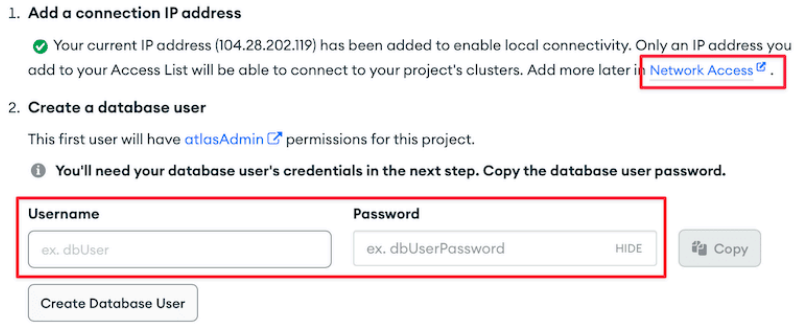
Let's say we make `TodoApp` project.

Create Cluster

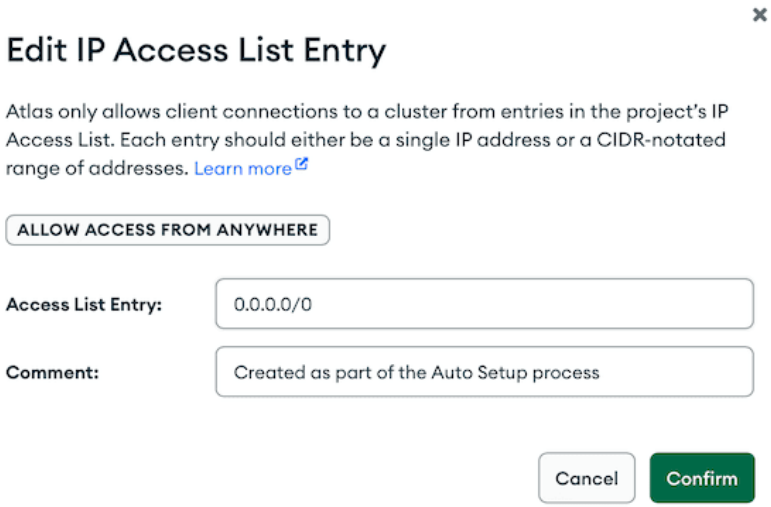
In the project, we can create cluster; choose free version.



Create user/password for the cluster: chos5/1234hello (example)

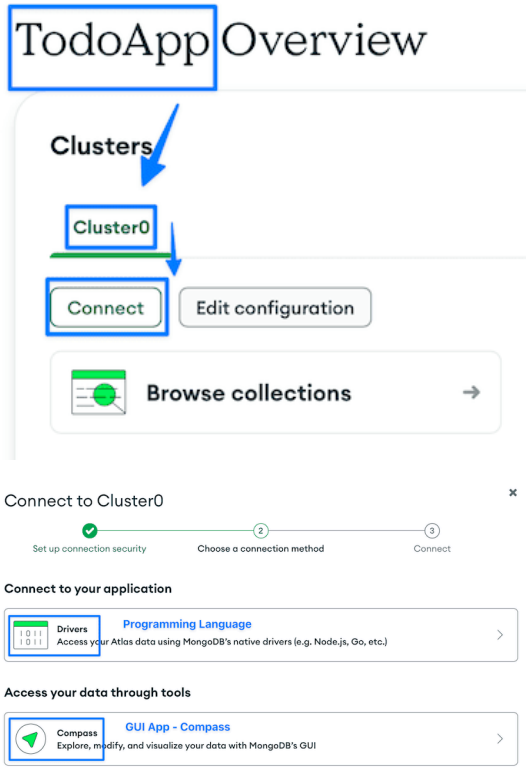


Edit to allow the access from anywhere for easy development.

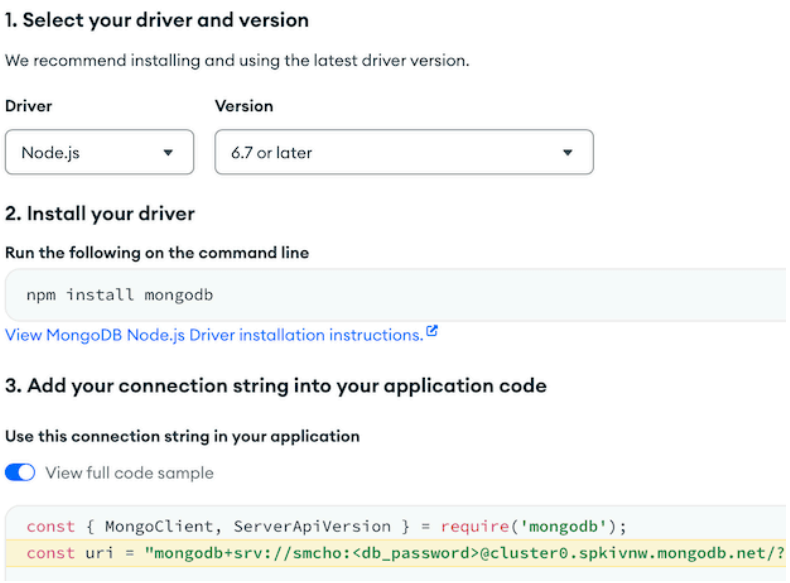


Connect to Cluster0

We assume that we use only cluster0 in the project; if you use other cluster name, adjust accordingly.



Choose `Node.js` to get the information to access the MongoDB.



With the user `chos5` and the password `1234hello` (example values), we can access the MongoDB cluster (`spkinwx`) as:

```
mongodb+srv://chos5:1234hello@cluster0.spkinwx.mongodb.net/
```

The `appName` parameter in a MongoDB connection string is optional. It identifies the application that connects to the database, which is useful for monitoring, logging, and debugging on the server side because MongoDB can see which application is making the requests.

```
mongodb+srv://chos5:1234hello@cluster0.spkinwx.mongodb.net/?appName=Cluster0
```

.env

We store `user/password` information in the `.env` file.

```
MONGO_USER=YOUR_USER
MONGO_PASSWORD=YOUR_PASSWORD
MONGO_CLUSTER=YOUR_CLUSTER
```

Use the env template:

1. Copy `env` to `.env` and replace your user/password/cluster with yours.
2. Be sure that `.gitignore` contains `.env` so it is not uploaded to GitHub.

You can add more information such as:

```
MONGO_DATABASE=YOUR_DATABASE
```

We use `dotenv` package to access the variables in the `.env` file.

```
require('dotenv').config({ quiet: true });
```

If the `.env` file is not in the directory where you run `node`, we need to specify its location (the path is relative to that directory).

```
require('dotenv').config({ path: './path/to/your/.env' });
```

`uri.js` Utility module

We use `uri.js` to access the environment variables and create the uri string for MongoDB access.

```
const dotenv = require("dotenv");

// Load environment variables
dotenv.config({quiet: true, path: "../.env",});

// Validate required variables
const user = process.env.MONGO_USER;
if (!user) throw new Error("MONGO_USER is not defined");
const password = process.env.MONGO_PASSWORD;
if (!password) throw new Error("MONGO_PASSWORD is not defined");
const cluster = process.env.MONGO_CLUSTER;
if (!cluster) throw new Error("MONGO_CLUSTER is not defined");
// Database name
const databasename = process.env.MONGO_DATABASE || "todoapp";

// Construct the MongoDB connection string
// encodeURIComponent: user names and passwords may contain special characters such as @ or /
const uri = `mongodb+srv://${encodeURIComponent(user)}:${encodeURIComponent(password)}@cluster0.${cluster}.mongodb.net/`;

// Export (CommonJS style)
module.exports = {uri, databasename,};
```

CommonJS Module Style

In this example, we use CommonJS module style:

1. It uses `require` for module import.

```
const dotenv = require("dotenv");
```

2. It uses `module.exports` for module export.

```
module.exports = {
  uri,
  databasename,
};
```

The MongoClient Object

With the `uri`, we can create the MongoDB client object to access the MongoDB database.

```
const { MongoClient } = require('mongodb');
const { uri } = require('./uri.js');
const client = new MongoClient(uri);
```

To make sure that your application uses only the commands and behaviors defined in a specific stable API version, we can use `ServerApiVersion`.

```
const { MongoClient, ServerApiVersion } = require('mongodb');
const { uri } = require('./uri.js');
const client = new MongoClient(uri, {
  serverApi: {
    version: ServerApiVersion.v1,
    strict: true,
    deprecationErrors: true,
  }
});
```

Connection Layer and Database

This is the mental model of the official `mongodb` driver:

```
const { MongoClient } = require("mongodb");

const client = new MongoClient(uri); // connection pool (not yet connected)

let db = null;

async function getDB(databaseName = DATABASE) {
  if (db) return db;           // already connected & using this DB

  await client.connect();      // open TCP connections, auth, etc.
  db = client.db(databaseName) // lightweight DB handle on that connection
  return db;
}
```

In this model:

- client = connection pool object (one per app is typical)
- db = logical database (a thin wrapper over client)

```
client  ---- sockets, pool, auth
   |
   +-- db("todoapp")
   +-- db("logs")
```

To close DB connection, we use client:

```
client.close()
```

`db.js` Utility function

We export the `client` object and the `getDB` function:

```
// db.js
// Create a single MongoClient instance
const client = new MongoClient(uri); // connection

// Create and reuse a single database connection
async function getDB(database = DATABASE) {
  if (db) return db;           // reuse existing connection

  await client.connect();      // connect once
  db = client.db(database);    // store db globally
  return db;
}

module.exports = {
  client,
  getDB,
  ...
};
```

Singleton Design Pattern

> In the code, we reuse `db` if it already holds a database: `if (db) return db;`.
> In this way, only one database object is used.
> This design pattern is called the **Singleton** pattern.

Collections

We need collections, so in the `db.js` module we write utility functions that return them.

```
const POSTS = 'posts';
const COUNTER = 'counter';
```

```
// Return the "posts" collection
function getPostsCollection() {
  if (!db) throw new Error("Database not connected. Call getDB() first.");
  return db.collection(POSTS);
}

// Return the "counter" collection
function getCounterCollection() {
  if (!db) throw new Error("Database not connected. Call getDB() first.");
  return db.collection(COUNTER);
}
```

Example1: ping.js

We can use the ping command to check if we can access the MongoDB.

To run it, copy `env` to `.env` (one directory above `mongodb`) and edit it. Then run `npm install` and `node ping.js` in the `mongodb` directory.

```
const {client, getDB} = require('./util/db.js')

async function run() {
  try {
    const db = await getDB("admin"); // get DB
    await db.command({ ping: 1 }); // ping
    console.log("Pinged your deployment. You successfully connected to MongoDB!");
  } finally {
    // Ensures that the client will close when you finish/error
    await client.close();
  }
}
run().catch(console.dir);
```

We use the `client` object that `db.js` exports to close the connection:

```
const { client, getDB } = require('./util/db.js');
```

In CommonJS, we cannot use top-level `await`, so we put the `await` code in an `async` function and run it.

```
async function run() {
  try { ... }
  finally { await client.close(); }
}
run().catch(console.dir);
```

Example 2: database.js: Create DB & Collection

MongoDB will:

- Create database nku_demo
- Create collection students
- Insert one document (thus materializing both)

```
// 1. Create (or use) a database
const db = await getDB("nku_demo"); // created only when data is added
console.log("Connected!");
// 2. Create (or use) a collection
const students = db.collection("students");
// 3. Optionally, define indexes or insert data
await students.insertOne({ name: "Alice", age: 22, major: "ASE" });
```

Optional: Explicitly create collection

If you want to explicitly create a collection (e.g., with schema validation, capped size, etc.) before inserting, use:

```
await db.createCollection("students", {
  validator: {
    $jsonSchema: {
      bsonType: "object",
      required: ["name", "age"],
      properties: {
        name: { bsonType: "string" },
        age: { bsonType: "int", minimum: 0 }
      }
    }
  }
});
console.log("Collection 'students' created with schema!");
```

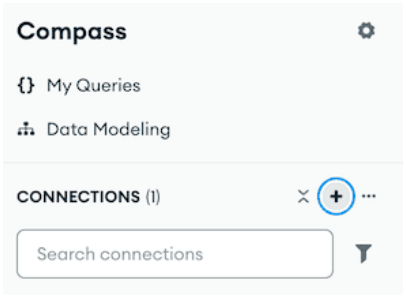
The Need for Mongoose

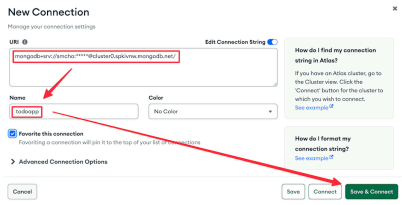
> In MongoDB, a schema is optional, and this sometimes causes problems.
>
> Mongoose solves this by enforcing a schema (a data model).
>
> We will discuss the idea of Mongoose in the next section.

Compass as the Desktop MongoDB Tool

We use the uri information to connect Compass (and many other applications).

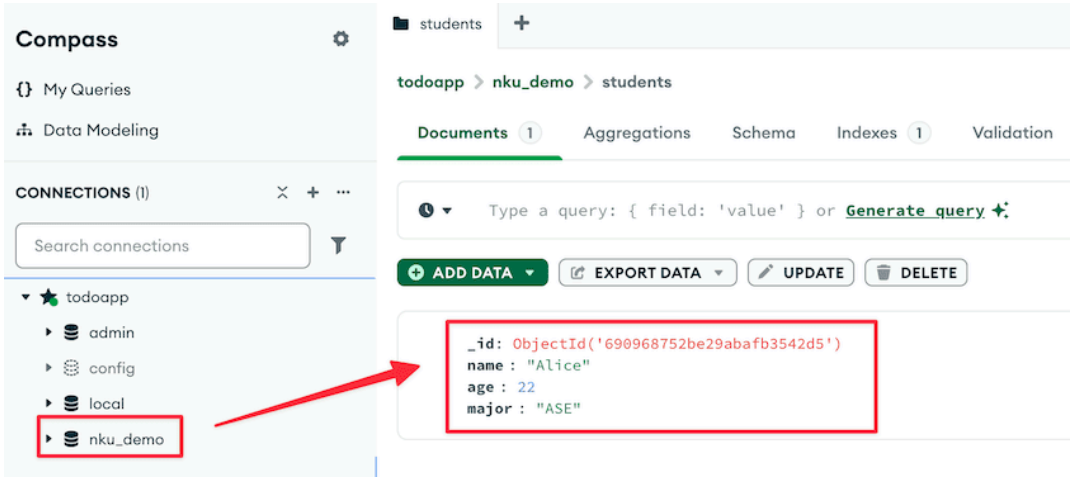
```
(Example, use your uri)
mongodb+srv://chos5:1234hello@cluster0.spkinwx.mongodb.net/
```



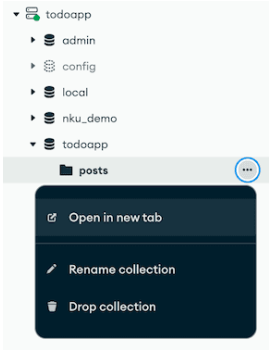


Verify your database & collection

You check the results:

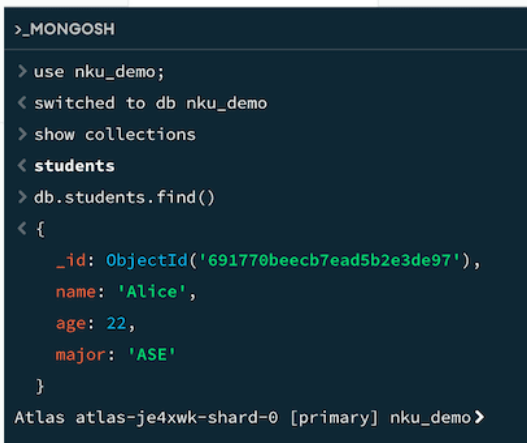


To refresh the screen, use the “Open in new tab” function.



You can also check it from the MongoDB shell in the Compass:

```
show dbs
use nku_demo
show collections
db.students.find()
```



Understanding MongoDB

Lazy Creation

In MongoDB, databases and collections are created lazily — they don’t exist until you actually insert data into them. So, when you run something like:

```
const db = client.db("mydb");
const coll = db.collection("users");
await coll.insertOne({ name: "Alice" });
```

MongoDB automatically creates both mydb and users if they don’t already exist.

The `_id` field

Every MongoDB document has a special field `_id` that uniquely identifies it. By default, MongoDB creates it as an ObjectId — a 12-byte binary value (not a string).

Example when you insert a document:

```
{
  _id: ObjectId("672746cbe3e9d79cfafca932"),
  title: "Hello",
  date: "2025-11-03"
}
```

convert string → ObjectId

When data comes from HTTP (forms, JSON, etc.)

In Express, everything you receive from a request — body, params, query — is a string.

Example:

```
console.log(req.body._id)  // "672746cbe3e9d79cfafca932"  ← string, not ObjectId
```

MongoDB cannot match a string against a stored ObjectId:

```
await db.collection('posts').deleteOne({ _id: "672746cbe3e9d79cfafca932" })
// ❌ No document will be deleted, because the types don't match
```

Solution: ObjectId

That's why we convert the string to an ObjectId:

```
const { ObjectId } = require('mongodb');
await db.collection('posts').deleteOne({ _id: new ObjectId(_id) });
```

Now the query compares ObjectId to ObjectId, and MongoDB finds the right document.

When you don’t need it

You only need `ObjectId()` if your `_id` values are ObjectIds (MongoDB’s default).

If you define `_id` as a plain number or string when inserting, then no conversion is needed:

```
await posts.insertOne({ _id: 1, title: 'A' });
await posts.deleteOne({ _id: 1 });  // works fine
```